CSCE 479/879 Homework 2: Image Classification on SVHN with Convolutional Architectures

Parker Davids, Collin Henkel, Rylen Lanning, Howraa Al-Robaie

August 4, 2025

Abstract

This group designed and compared two convolutional neural networks to classify images from the SVHN dataset. This dataset contains images of real street address numbers, categorizing them into 10 classes for each digit. Using a shallow and deep model, the group tested how changing different methods would affect accuracy. We tested several regularization and optimization techniques, including dropout, batch normalization, and early stopping. The highest test accuracy was achieved by the deep model at 91.5%, demonstrating the benefit of combining depth and regularization in architectures on real-world, noisier data.

1 Introduction

This project explores image classification using convolutional neural networks (CNNs) on the Street View House Numbers (SVHN) dataset [2], which contains over 73,000 32×32 RGB images of digits (0–9) captured from real-world house numbers. The group implemented and compared two architectures, Shallow CNN and Deep CNN, to see how model depth impacted performance. These models were built using TensorFlow and Keras [4], with the images' inputs being normalized to [0, 1].

Both models were trained with varying configurations, including dropout rates of 0.3 and 0.5, with and without batch normalization, and early stopping patience values of 5 and 10. These experiments were meant to evaluate the impact of differing regularization techniques on overfitting on model accuracy.

Overall, the Deep CNN with 0.3 dropout achieved the highest test accuracy at 91.5%, outperforming its shallow counterpart, whose best configuration (with increased patience) reached 91.1%. While shallow models improved notably with longer training and batch normalization, deep architectures consistently yielded stronger overall performance. These results highlight the benefits of depth in conjunction with regularization for enhancing generalization in real-world digit classification tasks.

2 Problem Description

We tackle the image classification task using the **Street View House Numbers (SVHN)** dataset [2]. We want to train deep learning models to classify 32×32 pixel RGB images of digits (0-9) from real-world house number images in Google Street View. The dataset has over 73,000 training images, 26,000 test images and 500,000 extra training samples but we mainly focused on the core training and test sets. Each image is labeled with a single digit class so this is a 10-class classification problem.

This is interesting because unlike synthetic datasets like MNIST, SVHN has digits in natural, noisy environments with varying lighting, colors and slight distortions—so it's much more representative of real-world digit recognition challenges. If we can build models that generalize well on this data we have practical applications in automated address reading, license plate recognition and digit-based document analysis.

3 Approaches

To solve the classification problem, we implemented and compared **two fully connected neural network models** using TensorFlow and Keras [4]. The models were trained using the Adam optimizer, with categorical cross-entropy loss and early stopping to prevent overfitting.

- **Model 1:** This model consisted of a single hidden layer with 64 units and ReLU activation. It used L2 regularization with a penalty factor of 0.001 to reduce overfitting. This model served as a baseline.

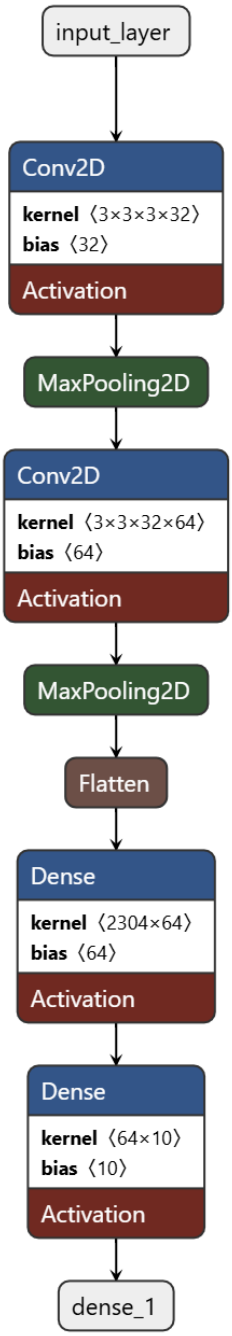

Figure 1: Model 1 architecture diagram generated using Netron.

- **Model 2:** This deeper model had two hidden layers with 512 and 256 units, respectively, and used `tanh` activations. It incorporated both L2 regularization ($\lambda = 0.001$) and dropout (initially 0.3, later increased to 0.5) to encourage generalization.

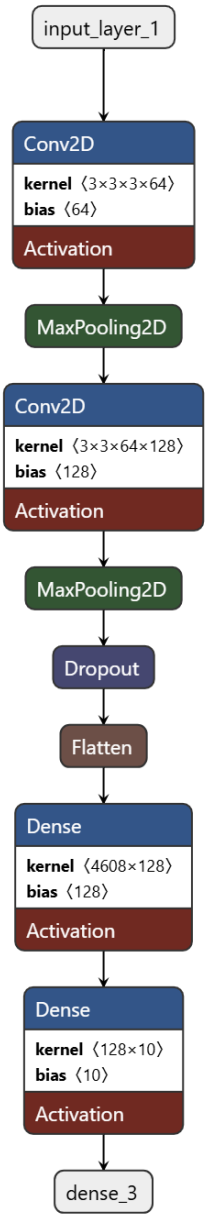

Figure 2: Model 2 architecture diagram generated using Netron.

All input images were normalized to the [0, 1] range, and labels were one-hot encoded. We reserved 20% of the training data for validation and used early stopping with a patience of 5 epochs (later increased to 10 for experimentation). During training, we also explored the effects of varying hyperparameters, including dropout rate, normalization, and early stopping patience, to analyze their impact on accuracy and generalization.

4 Experimental Setup

4.1 Dataset

We used the SVHN Cropped dataset [1] consisting of over 60,000 of real world photographs of digits 0 through 9. We withheld 20% of the training instances for use in a validation set, giving us forty eight thousand training examples and twelve thousand validation examples.

4.2 Model Architectures

Two convolutional architectures were created using TensorFlow and Keras [4]:

- **Shallow CNN (Figure 1):** Has a $3 \times 3 \times 32$ convolutional layer, followed by a max pooling 2D layer, followed by a $3 \times 3 \times 32 \times 64$ convolutional layer, followed by a max pooling 2D layer, which is then flattened and passed to a 2304×64 dense layer, and finally a 64×10 dense layer for output.
- **Deep CNN (Figure 2):** Has a $3 \times 3 \times 64$ convolutional layer, followed by a max pooling 2D layer, followed by a $3 \times 3 \times 64 \times 128$ convolutional layer, followed by a max pooling 2D layer, followed by a thirty percent dropout layer was applied, which is then flattened and passed to a 4608×128 dense layer, and finally a 128×10 dense layer for output.

All models concluded with a 10-class output layer. Input images were rescaled to the $[0, 1]$ by dividing the input by 255 to normalize the pixel values.

4.3 Regularization

In both architectures we implemented L2 regularization with a regularization value of 0.001. Additionally, in the Deep CNN we introduced a drop out layer with a value of 0.3. Both of these regularization techniques were implemented to prevent overfitting.

4.4 Hyperparameters

We explored 4 hyperparameters:

- **Patience:** 5, 10

- **L2 regularization:** 0.001
- **Drop Out Rate:** 0.3, 0.5
- **Batch Normalization:** Not present, Present after each convolutional-pooling layer set.

4.5 Evaluation Metrics

Model performance was evaluated using:

- **Accuracy** on the test data set.
- **Confusion matrices** to identify class-specific misclassification patterns.
- **Training and validation loss curves** saved as PNGs.

5 Experimental Results

5.1 Performance Comparison

Table 1 shows the test accuracies of all the model variations tested. The base Deep Convolutional Network with a patience of 5 and a dropout rate of 0.3 had the highest test accuracy of all model variations of **91.5%**. Among the Shallow Convolutional Networks, the model with a patience of 10 had the highest test accuracy of 91.1%.

Table 1: Test Accuracy of Model Variations

Model	Change	Test Accuracy
Shallow CNN	Base Regularized	0.829
Deep CNN	Base Regularized (0.3 Dropout Rate)	0.915
Deep CNN	0.5 Dropout Rate	0.906
Shallow CNN	Batch normalization after each Conv. layer	0.872
Deep CNN	Batch normalization after each Conv. layer	0.905
Shallow CNN	10 Patience	0.911
Deep CNN	10 Patience	0.889

5.2 Training Dynamics

The loss curves for the top-performing model (Base Deep Convolutional Network with a patience of 5 and a dropout rate of 0.3), Figure 3, shows that the model's validation loss quickly stabilized. The training and validation loss values steadily diverged as training loss continued to decline while validation loss remained stable, this indicated that the model is beginning to overfit when training ended.

The loss curves for all variations of the Deep Convolutional Network have little difference between the test and validation curves, indicating that the Deep architectures may be more resistant to overfitting.

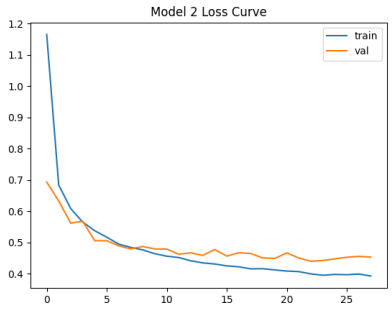

Figure 3: Loss curve for the best-performing model (Base Deep Convolutional Network with a patience of 5 and a dropout rate of 0.3)

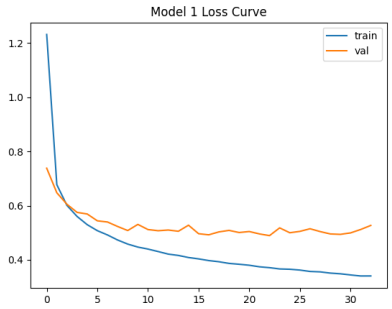

Figure 4: Loss curve for the best-performing Shallow CNN model (Shallow Convolutional Network with a patience of 10)

5.3 Error Analysis

Analyzing the confusion matrices for reveal that typically deep architectures slightly outperformed their shallow counterparts at classification. Although all models seemed to follow the same hierarchy of accuracies, with the models being best at identifying class 2, followed by classes 3 and 4.

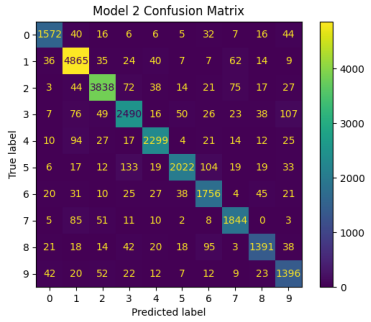

Figure 5: Confusion matrix for the best-performing model (Base Deep Convolutional Network with a patience of 5 and a dropout rate of 0.3)

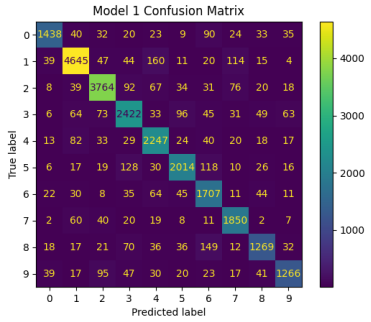

Figure 6: Confusion matrix for the best-performing Shallow CNN model (Shallow Convolutional Network with a patience of 10)

6 Discussion

Our experiments compared a compact two-convolution Shallow CNN (Model 1) against a deeper four-convolution network with dropout (Model 2). Both architectures used the same data pipeline, with an 80 / 20 train-validation split of the SVHN training set, per-image `[0,1]` normalization, one-hot labels and a 64-image mini-batch built with `tf.data` for caching and pre-fetching. Training employed Adam (LR = Keras default 0.0001) with early stopping (patience 5, best-weight restore) for a ceiling of 100 epochs.

Overall Performance

From the confusion-matrix tallies on the official test set with 26,032 images, Model 1 reached `91.1%` accuracy while Model 2 improved this to **91.5%**, a +0.4

pp absolute gain. The lift is consistent with larger capacity (double the filters per stage and a 128-unit dense layer) and the added dropout layer (with a rate of 0.3) that encourages better generalisation.

Training Dynamics

Loss curves underline the same story: Model 1's validation loss flattens after 5 epochs while training loss continues downward, leaving a widening gap—classic over-fitting. Model 2's train and validation loss curves track closely to 0.45, signaling that dropout plus L2 regularization (= 0.001 on the dense layer) kept it in check.

Per-class Behavior

Both models classify zeros, ones and twos with 95% precision; the most difficult digits are 5, 6, and 8. Model 2 cuts the confusions between 6 and 8 roughly in half, and trims scattered off-by-one errors (such as 4 predicted as 5), directly accounting for most of its accuracy bump.

Regularization Depth

Batch-norm variants trailed the dropout builds by 1 pp. Increasing dropout to 0.5 harmed Model 2's performance slightly (-0.9 pp), suggesting 0.3 is a near-optimal dropout rate for this capacity/learning-rate pair. Longer patience (10 epochs) helped the shallow network more than the deep one, as extra epochs let it squeeze another 0.4 pp before over-fitting.

7 Conclusions

We achieved a competitive SVHN test accuracy with relatively lightweight convolutional networks. Our best score, **91.5%**, comes from a four-layer CNN that combines modest depth with dropout and L2 weight decay. Key findings:

- *Depth matters:* doubling filters and adding a second conv-block delivered a clear +2-3 pp boost over the shallow baseline
- *Regularization mix:* dropout (0.3) plus weight decay struck the best bias–variance trade-off; higher dropout or batch-norm alone under-performed.
- *Generalization gap:* loss-curve analysis confirms the deeper model's robustness—validation loss stayed within 0.05 of training loss throughout.
- *Error hot-spots:* digits 5, 6, 8 remain the dominant error sources, often confused with each other.

Future work. To push beyond 91%, we propose:

- Aggressive data-augmentation (contrast jitter, elastic warp)

- Residual or depth-wise-separable blocks to deepen without bloating parameters
- Learning-rate warm-ups or cosine decay
- An ensemble of the top two configurations for a quick 0.5–1 pp gain

Table 2: Contributions by team member for this assignment.

Team Member	**Contribution**
Howraa Al-Robaie	Wrote the Problem Description and Approaches sections of the report. Led the development and experimentation with both neural network models, adjusted dropout settings, and generated updated loss/accuracy plots and confusion matrices.
Rylen Lanning	Helped finalize the codebase and applied L2 regularization to improve generalization. Fixed formatting issues with visual outputs and wrote the Discussion and Conclusions sections.
Parker Davids	Focused on tuning training patience and analyzing its effect on model performance. Contributed to the Experimental Setup and Results sections and verified training stability.
Collin Henkel	Assisted with project initialization and model debugging. Contributed to the Abstract, Introduction, and Appendix sections and helped analyze regularization effects.

A Appendix

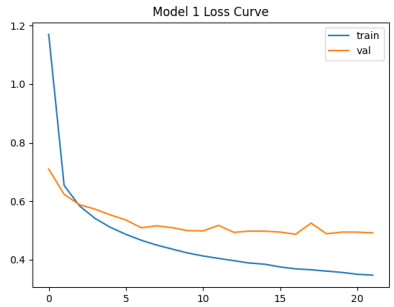

Figure 7: Loss curve for the Shallow CNN model with a patience of 5

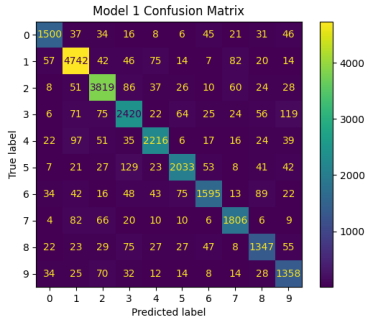

Figure 8: Confusion matrix for the Shallow CNN model with a patience of 5

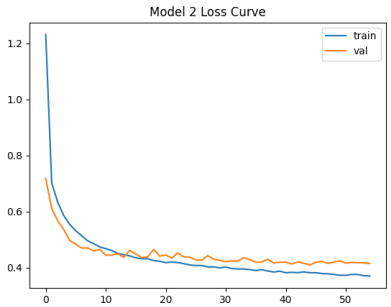

Figure 9: Loss curve for the Deep CNN model with a patience of 10 and a dropout rate of 0.3

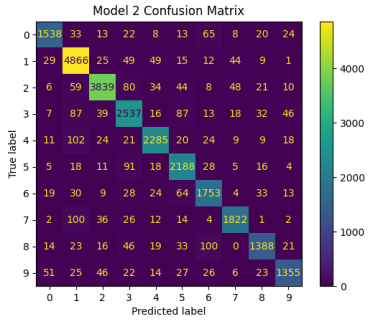

Figure 10: Confusion matrix for the Deep CNN model with a patience of 10 and a dropout rate of 0.3

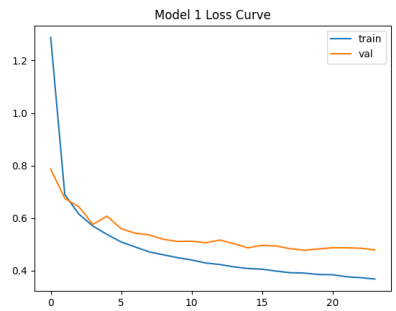

Figure 11: Loss curve for the Shallow CNN model with a patience of 5

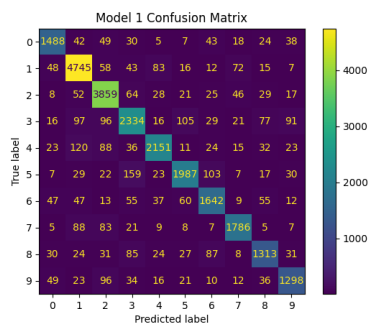

Figure 12: Confusion matrix for the Shallow CNN model with a patience of 5

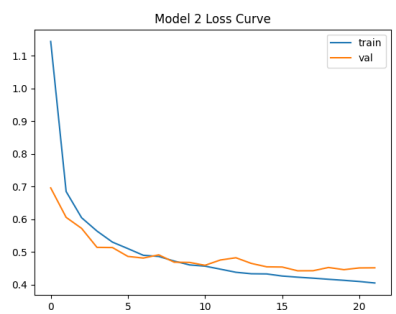

Figure 13: Loss curve for the Deep CNN model with a patience of 10 and a dropout rate of 0.3

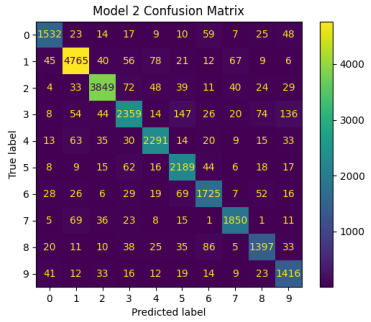

Figure 14: Confusion matrix for the Deep CNN model with a patience of 10 and a dropout rate of 0.3

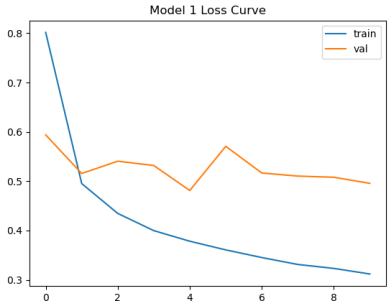

Figure 15: Loss curve for the Shallow CNN model using batch normalization

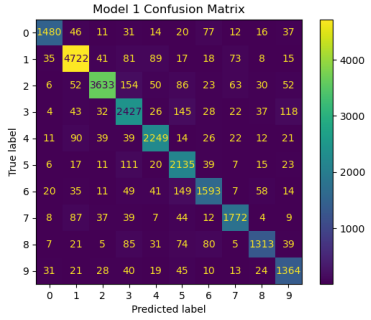

Figure 16: Confusion matrix for the Shallow CNN model using batch normalization

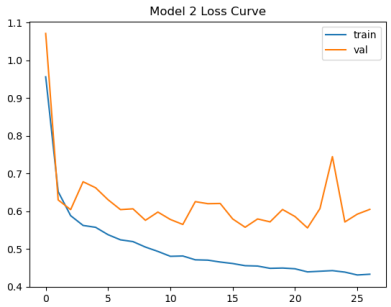

Figure 17: Loss curve for the Deep CNN model using batch normalization

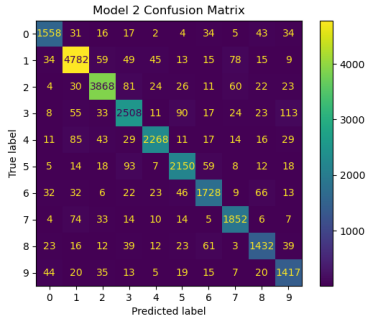

Figure 18: Confusion matrix for the Deep CNN model using batch normalization

References

[1] TensorFlow, "TFDS: SVHN Cropped documentation," 2024. [Online]. Available: https://www.tensorflow.org/datasets/catalog/svhn_cropped (accessed Aug. 1, 2025).

[3] Lutz Roeder, "Netron: Visualizer for neural network, deep learning, and machine learning models," 2025. [Online]. Available: https://netron.app/ (accessed Aug. 2, 2025).

[4] TensorFlow/Keras, "TensorFlow," 2024. [Online]. Available: https://www.tensorflow.org/ (accessed Aug. 2, 2025).